

# Acquisition et Nettoyage des Données

Master 1 Data & IA — 2025-2026

---

Ahmad ALMAKSOUR — Faculté de Gestion, Économie et Sciences

# Plan de la séance

**01**

Architecture  
interne de Pandas

**02**

Vectorisation

**03**

Reshaping & Tidy Data

**04**

Merge & Join

**05**

Types avancés

**06**

Introduction à Polars

# Pourquoi maîtriser Pandas en profondeur ?

---

**80 %**

du temps d'un data scientist est consacré  
à l'acquisition et au nettoyage des  
données

**10×**

l'écart de performance entre du code  
Pandas naïf et du code vectorisé  
optimisé

**60 %**

des bugs en ML viennent de données  
mal jointes ou mal typées en amont

# Series, DataFrame, Index - Les 3 piliers

DATAFRAME

| Index | col_A    | col_B | col_C |
|-------|----------|-------|-------|
| 0     | Paris    | 75    | 3200  |
| 1     | Lyon     | 69    | 2800  |
| 2     | Lille    | 59    | 2100  |
| 3     | Bordeaux | 33    | 2500  |

## Series

Tableau 1D typé avec un Index

## DataFrame

Collection de Series partageant le même Index. La structure principale. Chaque colonne peut avoir un type différent.

## Index

L'axe des lignes. Par défaut entiers (0, 1, 2...). Peut être une date, un code produit, etc.

# dtypes - Le type de chaque colonne détermine tout

| dtype Pandas   | Alias | Taille mémoire     | Exemple de valeur | Piège courant                           |
|----------------|-------|--------------------|-------------------|-----------------------------------------|
| int64          | int   | 8 octets/valeur    | 42                | Pas de NaN possible (→ float64)         |
| float64        | float | 8 octets/valeur    | 3.14              | NaN = float, attention aux comparaisons |
| object         | str   | ~50+ octets/valeur | 'Paris'           | Le dtype le plus gourmand en mémoire    |
| category       | —     | ~1 octet/valeur    | 'Paris'           | Idéal si < 50 valeurs uniques           |
| datetime64[ns] | —     | 8 octets/valeur    | 2023-01-15        | Fuseau horaire absent par défaut        |
| bool           | —     | 1 octet/valeur     | True / False      | NaN => object silencieusement           |

# Quel sera le dtype de chaque colonne ?

---

Sans exécuter le code, prédir le dtype Pandas de chaque colonne. Justifiez votre réponse.

```
import pandas as pd

df = pd.DataFrame({
    'id'      : [1, 2, 3, np.NaN],          # → dtype ?
    'ville'   : ['Paris', 'Lyon', 'Lille', 'Rennes'], # → dtype ?
    'prix'    : ['100€', '200€', '150€', '180€'],    # → dtype ?
    'actif'   : [True, False, True, None],          # → dtype ?
    'date'    : ['2023-01', '2023-02', '2023-03', '2023-04'], # → dtype ?
})
```

# Copy vs View — Le piège qui corrompt vos données

## ✗ Comportement dangereux

```
# Sélection par filtrage = VIEW ou COPY ?  
subset = df[df['ville'] == 'Paris']  
subset['prix'] = 0  
# df est-il modifié ? Peut-être. Ça dépend.
```

## ✓ Toujours utiliser .copy()

```
# Copie explicite = comportement garanti  
subset = df[df['ville'] == 'Paris'].copy()  
subset['prix'] = 0 #df intact  
# Modifie uniquement subset, jamais df
```

### Qu'est-ce qu'une View ?

Une fenêtre sur les données originales.  
Modifier une View peut modifier le DataFrame source - ou non. Pandas ne garantit rien.

### Qu'est-ce qu'une Copy ?

Une copie indépendante en mémoire. Modifier une Copy ne touche jamais le DataFrame source. Comportement 100% prévisible.

# Que va afficher ce code ?

---

Prédisez df['prix'].sum() après exécution complète. Sans lancer le code.

```
df =  
pd.DataFrame({'ville':['Paris','Lyon','Lille'],'prix':[300,200,150]})  
  
# Bloc A  
sous = df[df['prix'] > 100]  
sous['prix'] = 0  
  
# Bloc B  
sous2 = df[df['prix'] > 100].copy()  
sous2['prix'] = 0  
  
print(df['prix'].sum())    # → ???
```

Après Bloc A seul

Après Blocs A+B



# Opérations vectorisées - Pourquoi les boucles Python sont bannies

Python est un langage interprété. Chaque itération d'une boucle for a un coût fixe élevé. Pandas délègue le calcul à NumPy, qui exécute du code C compilé sur des blocs mémoire contigus.

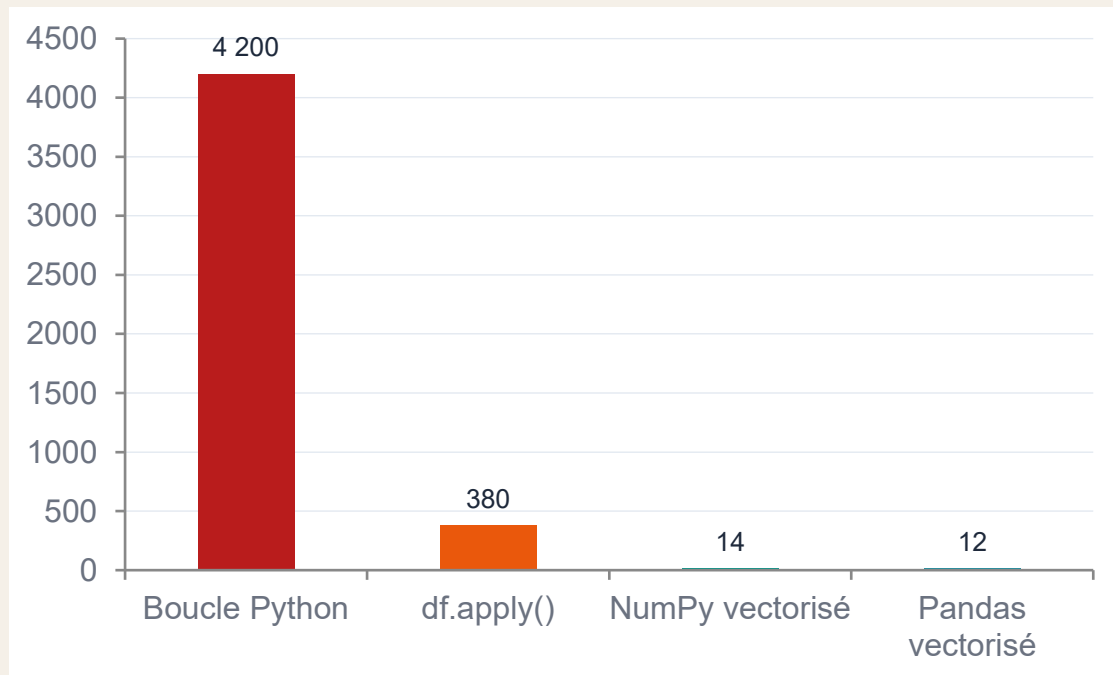
```
# BOUCLE - 1 000 000 lignes : ~4 secondes
prices = []
for i in range(len(df)):
    prices.append(df.iloc[i]['prix_brut'] * 1.20)
df['prix_ttc'] = prices
```

```
# VECTORISÉ - 1 000 000 lignes : ~12 ms

df['prix_ttc'] = df['prix_brut'] * 1.20
# 333x plus rapide. Même résultat.
```

Principe : une opération sur une colonne entière = une seule instruction envoyée au processeur via C. NumPy gère l'itération en bas niveau. Python ne voit que le résultat.

# Benchmark - Comparaison des approches sur 1M lignes



**4200 ms**

Boucle for Python  
Interpréteur Python appel par appel

**380 ms**

df.apply() par ligne  
Toujours une boucle Python déguisée

**14 ms**

NumPy vectorisé  
C compilé, opérations sur blocs mémoire

**12 ms**

Pandas vectorisé  
Même moteur NumPy, syntaxe orientée données

# apply() - Quand l'utiliser (et quand l'éviter)

## ✗ Cas où apply() est inutile — vectorisez à la place

```
# Calcul arithmétique simple : éviter apply()
df['ttc'] = df['ht'].apply(lambda x: x * 1.20)
# Écrire simplement : df['ttc'] = df['ht'] * 1.20
```

## ✓ Cas légitimes pour apply()

```
# Logique conditionnelle complexe
def categorize(row):
    if row['age'] < 25 and row['revenu'] > 3000:
        return 'jeune_aisé'
    return 'standard'
df['segment'] = df.apply(categorize, axis=1)
```

## Alternatives vectorisées à connaître absolument

### np.where()

```
df['cat'] = np.where(df['prix'] > 100, 'premium',
                    'standard')
# Équivalent vectorisé du if/else
```

### str.methods

```
df['ville'] = df['ville'].str.upper()
df['code'] = df['ref'].str[:3]
```

### np.select()

```
df['zone'] = np.select(
    [df['dep']=='75', df['dep']=='69'],
    ['Paris', 'Lyon'], default='Autre')
```

### .map(dict)

```
mapping = {'F': 'Femme', 'M': 'Homme'}
df['genre'] = df['sexe'].map(mapping)
```

# groupby() + agg() — La puissance du SQL dans Python

```
# Syntaxe de base — équivalent SQL GROUP BY
df.groupby('type_local')['valeur_fonciere'].mean()

# Agrégations multiples — une seule passe
stats = df.groupby(['type_local', 'departement']).agg(
    n_ventes    = ('valeur_fonciere', 'count'),
    prix_moy    = ('valeur_fonciere', 'mean'),
    prix_med    = ('valeur_fonciere', 'median'),
    prix_max    = ('valeur_fonciere', 'max'),
)

# transform() — conserver la taille d'origine
df['prix_norm'] = df.groupby('ville')['valeur_fonciere'].transform(
    lambda x: (x - x.mean()) / x.std())
```

## Split

Groupby divise le DataFrame en groupes selon la/les clé(s).

## Apply

La fonction d'agrégation est appliquée sur chaque groupe indépendamment.

## Combine

Les résultats sont réassemblés en un nouveau DataFrame.

## transform()

Comme agg(), mais retourne un résultat de même taille que df utile pour normaliser.

# transform() vs agg() : la différence clé

```
df =  
pd.DataFrame({'ville':['Paris','Paris','Lyon','Lyon'],'prix':  
[300,500,200,400]})
```

```
# agg() → un résultat par groupe (shape réduit)  
res = df.groupby('ville')['prix'].mean()  
# Lyon      300.0  
# Paris     400.0 ← 2 lignes seulement
```

```
# transform() → même shape que df original  
df['prix_moy'] =  
df.groupby('ville')['prix'].transform('mean')
```

```
#   ville  prix  prix_moy  
#   Paris   300    400.0  
#   Paris   500    400.0 ← répété  
#   Lyon   200    300.0  
#   Lyon   400    300.0 ← répété
```

```
df['ecart'] = df['prix'] - df['prix_moy']
```

Règle : agg() pour un rapport agrégé. transform() pour ajouter une colonne calculée par groupe dans le DataFrame original => indispensable pour les comparaisons relatives.

# agg() ou transform() ?

---

## Mission A

Produire un tableau du prix médian par type de bien pour un rapport PowerBI.

## Mission B

Ajouter une colonne `flag_cher` : True si le prix est supérieur à la médiane de la commune.

## Mission C

Calculer le Z-score du prix par rapport aux biens de même type dans le même département.

## Mission D

Obtenir le nombre de transactions par commune pour les marchés les plus actifs.

# Tidy Data - Le standard que tout data engineer doit connaître

**Hadley Wickham (2014)** a formalisé ce que les praticiens faisaient intuitivement. Un dataset « tidy » est structuré pour être analysé, visualisé et modélisé sans transformation préalable.

## Règle 1

**Chaque variable  
occupe une colonne**

---

*Ville, Prix, Type = 3 colonnes distinctes*

## Règle 2

**Chaque observation  
occupe une ligne**

---

*Une ligne = une vente immobilière unique*

## Règle 3

**Chaque type d'observation  
occupe un tableau**

---

*DVF et données météo = 2 tableaux séparés*

# melt() - Passer du format Wide au format Long (Tidy)

## AVANT — Format Wide (non tidy)

| Ville | Jan  | Fév  | Mars |
|-------|------|------|------|
| Paris | 3200 | 3250 | 3180 |
| Lyon  | 2800 | 2820 | 2790 |

→  
melt()

## APRÈS — Format Long (tidy) ✓

| Ville | mois | prix_m2 |
|-------|------|---------|
| Paris | Jan  | 3200    |
| Paris | Fév  | 3250    |
| Paris | Mars | 3180    |
| Lyon  | Jan  | 2800    |
| Lyon  | Fév  | 2820    |
| Lyon  | Mars | 2790    |

```
df_long = df_wide.melt(  
    id_vars = ['Ville'],  
    value_vars = ['Jan', 'Fév', 'Mars'],  
    var_name = 'mois',  
    value_name = 'prix_m2',  
)
```



# Ce dataset est-il tidy ?

Examinez ce tableau. Identifiez les violations des 3 règles de Wickham et proposez la structure correcte.

| Commune   | Prix_Jan_APT | Prix_Jan_MAI | Prix_Fev_APT | Prix_Fev_MAI | Pop_2020 |
|-----------|--------------|--------------|--------------|--------------|----------|
| Lille     | 3 200        | 2 800        | 3 150        | 2 900        | 234 000  |
| Roubaix   | 2 100        | 1 900        | 2 050        | 1 950        | 98 000   |
| Dunkerque | 2 400        | 2 200        | 2 380        | 2 150        | 87 000   |

## Questions :

1. Quelles règles de Wickham sont violées ?
2. Combien de colonnes aura le dataset tidy ? Combien de lignes ?
3. Quelle fonction Pandas permet de passer au format tidy ?
4. Pourquoi la colonne Pop\_2020 pose-t-elle un problème supplémentaire ?

# explode() — Normaliser les colonnes listes

Problème récurrent en data engineering : une colonne contient des listes (typique des APIs JSON). `explode()` transforme chaque élément de la liste en ligne distincte.

## AVANT - JSON dénormalisé

| produit | tags                          |
|---------|-------------------------------|
| Laptop  | ['tech', 'bureau', 'premium'] |
| Stylo   | ['bureau', 'papeterie']       |

```
# Normaliser en une ligne
df_norm = df.explode('tags')

df['tags_str'] = df['tags_str'].str.split(', ')
df_norm = df.explode('tags')
```

## APRÈS — Format normalisé ✓

| produit | tags      |
|---------|-----------|
| Laptop  | tech      |
| Laptop  | bureau    |
| Laptop  | premium   |
| Stylo   | bureau    |
| Stylo   | papeterie |

# Merge - Les 4 types de jointures

Comprendre les jointures est fondamental. En data engineering, une mauvaise jointure peut multiplier vos lignes silencieusement ou faire disparaître des données sans avertissement.

## INNER JOIN

Seulement les clés présentes dans les DEUX tables. Risque : perte de données si les clés ne matchent pas parfaitement.

## RIGHT JOIN

Toutes les lignes de droite. Peu utilisé , on préfère inverser l'ordre et faire un LEFT JOIN.

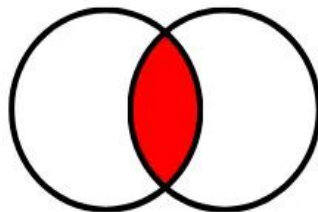
## LEFT JOIN

Toutes les lignes de gauche, NaN pour les colonnes droites sans correspondance. Le plus courant en data engineering.

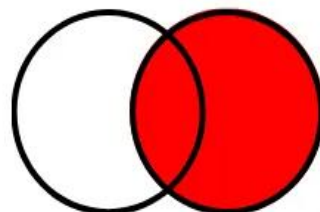
## OUTER JOIN

L'union de tout : toutes les lignes des deux tables, NaN là où il n'y a pas de correspondance.

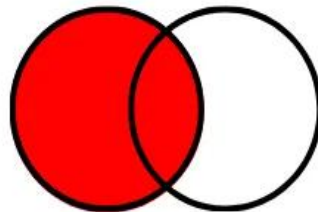
INNER JOIN



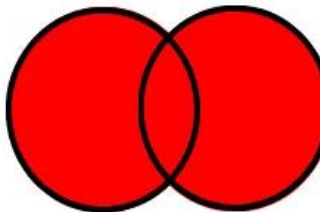
RIGHT JOIN



LEFT JOIN



FULL JOIN



# Merge - Clés multiples, suffixes et diagnostics

```
# Jointure sur clés multiples
result = pd.merge(
    df_ventes, df_produits,
    on = ['categorie', 'region'], # clé composée
    how = 'left',
    suffixes = ('_vente', '_ref'), # évite les collisions de noms
)

# Diagnostic post-jointure – à faire SYSTÉMATIQUEMENT
print(f'Avant : {len(df_ventes)} lignes')
print(f'Après : {len(result)} lignes')
# Si après > avant → doublon dans df_produits → à investiguer

# Vérifier les NaN introduits (lignes sans correspondance)
print(result.isnull().sum())
```

## Cardinalité 1:1

1 ligne à gauche = 1 ligne à droite. Résultat stable.  
Vérifier avec `df.duplicated(subset='cle')`.

## Cardinalité 1:N

1 ligne à gauche = N lignes à droite. Résultat  $\times N$ . Le cas le plus courant - intentionnel ou bug.

## Diagnostic obligatoire

Toujours comparer `len(avant)` et `len(après)`. Une multiplication silencieuse corrompt tout calcul agrégé.

# Diagnostiquer le résultat d'une jointure

```
df_ventes = pd.DataFrame({'id_produit':[1,2,3,4], 'montant':[100,200,300,400]})

df_ref = pd.DataFrame({
    'id_produit': [1, 2, 2, 3, 99], # id=2 doublon, id=99 absent
    'categorie' : ['A','B','B','C','D'],
})

result = pd.merge(df_ventes, df_ref, on='id_produit', how='left')
```

## Questions :

1. Combien de lignes a result ? Pourquoi != 4 ?
2. Quelle ligne de df\_ventes génère NaN ? Quel champ ?
3. Comment corriger df\_ref avant la jointure pour éviter la multiplication ?

# merge() vs join() vs concat() — Quand utiliser quoi ?

| Fonction           | Axe                | Clé de jointure           | Cas d'usage typique               | Équivalent SQL |
|--------------------|--------------------|---------------------------|-----------------------------------|----------------|
| <b>pd.merge()</b>  | Colonnes           | Colonnes explicites (on=) | Joindre 2 tables sur clé métier   | JOIN ON        |
| <b>df.join()</b>   | Index              | Index (par défaut)        | Joindre sur l'index - plus rapide | JOIN ON index  |
| <b>pd.concat()</b> | Lignes ou colonnes | Aucune , empilement       | Empiler des DataFrames identiques | UNION ALL      |

```
# merge() – jointure explicite sur colonne
pd.merge(df1, df2, on='id_produit', how='left')

# join() – jointure sur l'index (plus rapide)
df1.set_index('id').join(df2.set_index('id'), how='left')

# concat() – empilement vertical (axis=0) ou horizontal (axis=1)
pd.concat([df_jan, df_fev, df_mars], axis=0, ignore_index=True)
```

# datetime64 - Dates, heures et fuseaux horaires

```
# Parsing de dates au chargement
df = pd.read_csv('data.csv', parse_dates=['date_mutation'])

# Conversion manuelle
df['date'] = pd.to_datetime(df['date_str'], format='%d/%m/%Y')
df['date'] = pd.to_datetime(df['date_str'], infer_datetime_format=True)

# Extraction de composantes
df['annee'] = df['date'].dt.year
df['mois'] = df['date'].dt.month
df['jour'] = df['date'].dt.day_of_week # 0=Lundi
df['trimestre'] = df['date'].dt.quarter

# Fuseau horaire (timezone-aware)
df['ts_utc'] = df['ts'].dt.tz_localize('UTC')
df['ts_paris'] = df['ts_utc'].dt.tz_convert('Europe/Paris')

# Arithmétique temporelle
df['duree'] = df['date_fin'] - df['date_debut'] # timedelta
df['j+30'] = df['date'] + pd.Timedelta(days=30)
```

## Toujours parser au chargement

Les colonnes restées en 'object' bloquent toute opération temporelle. Vérifier avec `df.dtypes`.

## Attention aux formats mixtes

Si les dates ne sont pas toutes au même format, `to_datetime()` avec `errors='coerce'` retourne NaT (Not a Time).

## Fuseaux horaires en prod

Stocker en UTC, convertir pour l'affichage. Ne jamais comparer une date tz-aware avec une tz-naïve.

## Period vs Timestamp

Period = intervalle (ex: 'Janvier 2023'). Timestamp = instant précis. Choisir selon le grain de données.

# Polars - Pourquoi le secteur data y migre

Polars est une bibliothèque DataFrame écrite en Rust, construite sur Apache Arrow. Elle adresse les limitations fondamentales de Pandas pour les grands volumes.

| Limitation Pandas                    | Solution Polars                                    |
|--------------------------------------|----------------------------------------------------|
| Single-threaded - utilise 1 cœur CPU | Multi-threaded natif - tous les cœurs en parallèle |
| Mémoire : copies fréquentes          | Apache Arrow : zero-copy, colonnaire en mémoire    |
| Pas de lazy evaluation               | Moteur lazy : n'exécute que ce qui est utile       |
| Lent sur > 1M lignes                 | 5 à 50× plus rapide sur grands datasets            |
| API historique incohérente           | API cohérente et expressions expressives           |

Polars ne remplace pas Pandas - il le complète. En M1, vous maîtrisez d'abord Pandas. En pratique industrielle : Pandas pour l'exploration, Polars pour la production à grande échelle.



# Polars - Lazy evaluation et optimisation du plan d'exécution

## Eager (Pandas) — Exécution immédiate

```
# Chaque opération déclenche un calcul
df1 = df.filter(pl.col('prix') > 1000) # calcul
df2 = df1.select(['ville', 'prix'])    # calcul
df3 = df2.groupby('ville').agg(...)    # calcul
# 3 passes sur les données = lent
```

## Lazy (Polars) — Exécution différée

```
# On construit un plan – rien n'est exécuté
(df.lazy()
 .filter(pl.col('prix') > 1000)
 .select(['ville', 'prix'])
 .groupby('ville').agg(...))
.collect() # ← exécution ici seulement
```

Ce que fait le moteur Polars avant d'exécuter :

### 1. Analyse du plan

Construction d'un arbre logique représentant toutes les transformations.

### 2. Optimisation

Réorganisation : pousser les filtres en amont (predicate pushdown), éliminer les colonnes inutiles (projection pushdown).

### 3. Parallélisation

Découpe les opérations en tâches exécutées en parallèle sur tous les cœurs CPU.

### 4. Exécution

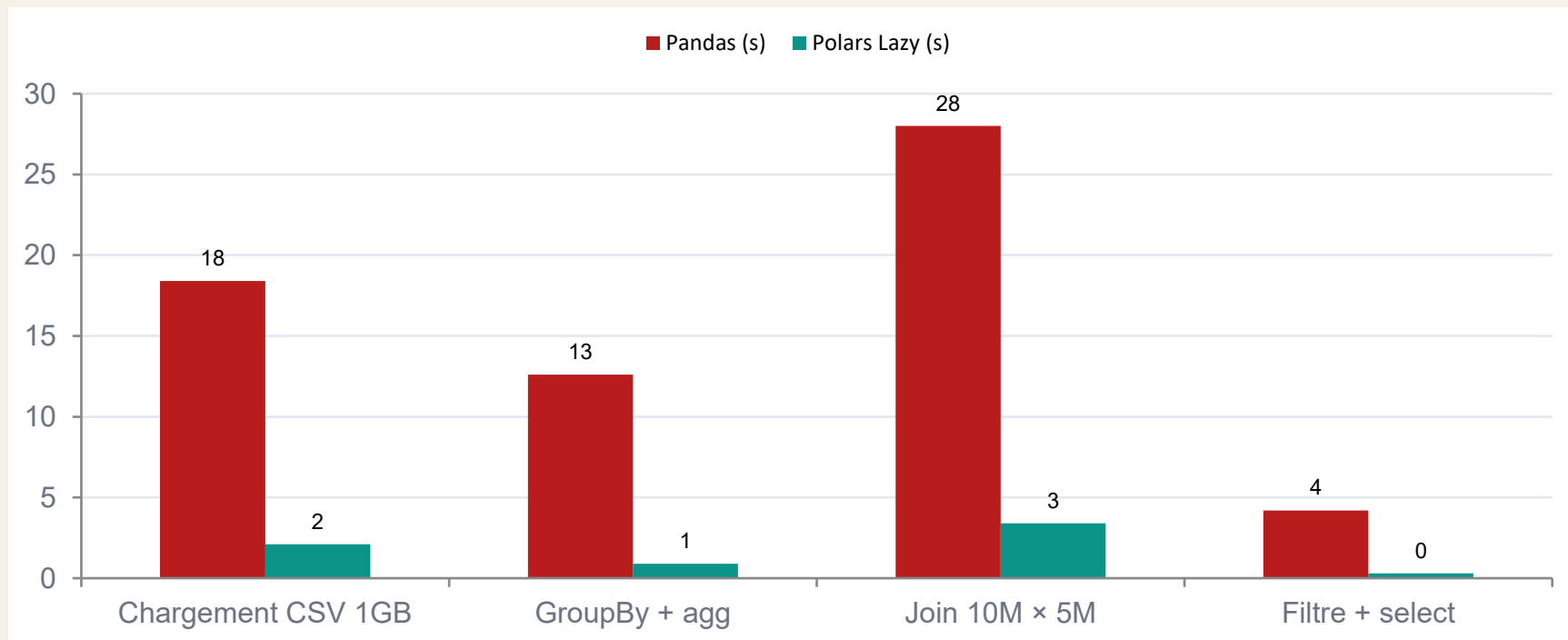
.collect() déclenche l'exécution du plan optimisé - une seule passe sur les données.

# Pandas vs Polars — Correspondances syntaxiques

| Opération          | Pandas                                          | Polars (Lazy)                                                 |
|--------------------|-------------------------------------------------|---------------------------------------------------------------|
| Chargement CSV     | <code>pd.read_csv('f.csv')</code>               | <code>pl.scan_csv('f.csv')</code>                             |
| Filtrage           | <code>df[df['prix'] &gt; 100]</code>            | <code>df.filter(pl.col('prix') &gt; 100)</code>               |
| Sélection colonnes | <code>df[['a', 'b']]</code>                     | <code>df.select(['a', 'b'])</code>                            |
| Nouvelle colonne   | <code>df['ttc'] = df['ht'] * 1.2</code>         | <code>df.with_columns((pl.col('ht')*1.2).alias('ttc'))</code> |
| GroupBy + agg      | <code>df.groupby('v').agg({'p': 'mean'})</code> | <code>df.groupby('v').agg(pl.col('p').mean())</code>          |
| Tri                | <code>df.sort_values('prix')</code>             | <code>df.sort('prix')</code>                                  |
| Null check         | <code>df['p'].isna()</code>                     | <code>pl.col('p').is_null()</code>                            |
| Exécution          | <code>(implicite)</code>                        | <code>.collect()</code>                                       |

La logique est identique seule la syntaxe diffère. Maîtriser Pandas vous permet d'apprendre Polars en quelques heures.

# Benchmark — Pandas vs Polars sur opérations réelles



# Pandas ou Polars en production ?

---

Polars est plus rapide. Mais ce n'est pas toujours la bonne réponse. Argumentez votre choix pour chaque situation.

## Exploration rapide

5 min de travail, 50 000 lignes, dans un Jupyter notebook d'analyse exploratoire.

## Pipeline ETL nuitier

50 millions de transactions par jour, résultat chargé dans un Data Warehouse.

## Projet d'équipe

6 data scientists, la moitié ne connaît pas Polars. Délai de livraison : 2 semaines.

## Rapport one-shot

Un manager demande une analyse d'un fichier Excel de 10 000 lignes.

# Pandas en production — Les règles d'or

01

## Vérifier les types au chargement

`df.dtypes` / `df.info(memory_usage='deep')` — toujours la première cellule de votre notebook.

03

## Vectoriser, jamais boucler

Toute opération sur une colonne doit être vectorisée. `apply()` par ligne = dernier recours documenté.

05

## Tester sur un échantillon d'abord

`df.sample(10_000)` pour explorer. La pipeline complète ne tourne qu'une fois validée sur l'échantillon.

02

## Copier avant de transformer

`df.copy()` dès qu'on fait un slicing + assignation. Sinon : `SettingWithCopyWarning` imprévisible.

04

## Documenter chaque transformation

Une cellule Markdown par bloc de transformations : pourquoi, pas comment. Le 'quoi' se lit dans le code.

06

## Choisir le bon format de sortie

CSV pour la lisibilité humaine, Parquet pour la performance. Jamais de CSV > 500 MB en production.

# Method Chaining - Écrire du code Pandas lisible et maintenable

Le method chaining consiste à enchaîner les opérations Pandas en une seule expression. C'est le style adopté par tous les data engineers professionnels.

## ❌ Style impératif — difficile à déboguer

```
df1 = df.dropna(subset=['prix','type'])
df2 = df1[df1['prix'] > 100]
df3 = df2.copy()
df3['ttc'] = df3['prix'] * 1.20
df4 = df3.rename(columns={'prix':'prix_ht'})
df5 = df4.sort_values('ttc', ascending=False)
# Variables intermédiaires : pollution, mémoire gaspillée
```

## ✅ Method chaining — propre et lisible

```
result = (
    df
    .dropna(subset=['prix', 'type'])
    .query('prix > 100')
    .assign(ttc=lambda d: d['prix'] * 1.20)
    .rename(columns={'prix': 'prix_ht'})
    .sort_values('ttc', ascending=False)
)
```

### **.assign()**

Crée de nouvelles colonnes sans .copy() préalable. Indispensable dans une chaîne.

### **.query()**

Filtre avec une string SQL-like. Plus lisible que df[df['col'] > x] dans une chaîne.

# Cas pratique - Jointure sur clé imparfaite

Problème réel : vous avez deux datasets sur les communes françaises. Les noms de communes ne sont pas formatés identiquement. Comment les joindre ?

```
# Source 1 : DVF - noms en majuscules, accents, tirets
df_dvf['commune'].head() # ['PARIS', 'AIX-EN-PROVENCE', 'SAINT MALO']

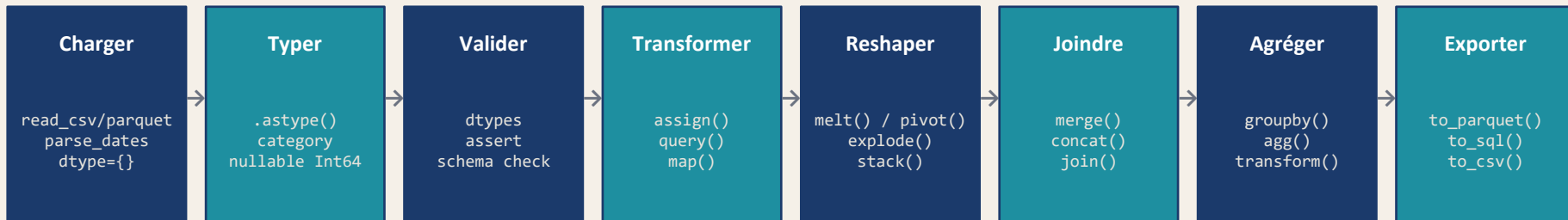
# Source 2 : Population INSEE - noms normalisés
df_pop['nom'].head()     # ['Paris', 'Aix-en-Provence', 'Saint-Malo']

# Étape 1 : normaliser les deux sources
import unicode
def normalize(s):
    return unicode.unidecode(str(s)).upper().strip().replace(' ', '-')

df_dvf['cle'] = df_dvf['commune'].apply(normalize)
df_pop['cle'] = df_pop['nom'].apply(normalize)

# Étape 2 : jointure + diagnostic
result = pd.merge(df_dvf, df_pop, on='cle', how='left')
miss = result['population'].isna().mean() * 100
print(f'{miss:.1f}% des communes sans correspondance')
```

# Synthèse - La chaîne complète de manipulation Pandas



## NumPy

Opérations numériques bas niveau, broadcasting, opérations matricielles

## ydata-profiling

Rapport qualité automatique, à lancer en début d'exploration

## Polars

Quand Pandas est trop lent (> 5M lignes ou opérations répétées en production)

## Parquet + DuckDB

Format de stockage + SQL analytique, combinaison gagnante en prod



# Récapitulatif - Ce qu'on a vu aujourd'hui

## Architecture Pandas

01

Series / DataFrame / Index — types, mémoire, copy vs view

## Reshaping

03

Tidy Data — melt(), pivot(), stack(), explode() — wide ↔ long

## Types avancés

05

datetime timezone-aware

## Vectorisation

02

Pourquoi les boucles Python sont bannies — 333× plus lent que vectorisé

## Merge & Join

04

4 types de jointures, clés multiples, diagnostic post-jointure systématique

## Polars

06

Lazy evaluation, moteur Rust, 5-10× plus rapide — complément de Pandas

# Questions ?

---

[ahmad.almosaalmaksour@lacatholille.fr](mailto:ahmad.almosaalmaksour@lacatholille.fr)